

1. Experiment Introduction

1.1 Experiment Objectives

- (1) Deeply understand the meaning of machine learning and the basic workflow for solving machine learning tasks;
- (2) Learn to use Python programming language and machine learning libraries (e.g., scikit-learn) for data processing, model building, model evaluation, and result visualization;
- (3) Understand the basic concepts and principles of K-means clustering model;
- (4) Understand the evaluation metrics for K-means clustering model (Silhouette Score, Adjusted Rand Index);
- (5) Master how to use K-means model to solve clustering tasks in machine learning.

1.2 Experiment Content

This experiment belongs to the cross-disciplinary analysis of wine science and machine learning. Different wine classes have differences in their chemical components. The chemical analysis dataset used in this experiment contains data on the main chemical components of different wine classes. The data information is as follows:

- **Column 1 (target):** `Wine` (葡萄酒类别)
- **Column 2:** `Alcohol` (酒精含量)
- **Column 3:** `Malic.acid` (苹果酸含量)
- **Column 4:** `Ash` (灰分含量)
- **Column 5:** `Alc` (灰分碱度)
- **Column 6:** `Mg` (镁含量)
- **Column 7:** `Phenols` (总酚含量)
- **Column 8:** `Flavanoids` (类黄酮含量)
- **Column 9:** `Nonflavanoid.phenols` (非类黄酮酚类含量)
- **Column 10:** `Proanth` (原花青素含量)
- **Column 11:** `Color.int` (颜色强度)
- **Column 12:** `Hue` (色调)
- **Column 13:** `OD` (光密度)
- **Column 14:** `Proline` (脯氨酸含量)

1.3 Experiment Principle

Clustering is an unsupervised learning method that aims to partition samples in a dataset into several clusters, such that samples within the same cluster are similar, while samples from different clusters are significantly different. Common clustering algorithms include K-means, hierarchical clustering, and DBSCAN. In this experiment, we will use the K-means algorithm for clustering analysis.

The basic idea of the K-means algorithm is as follows:

- (1) Randomly select K initial cluster centers.
- (2) Assign each sample to the nearest cluster center.
- (3) Recompute the center point of each cluster.
- (4) Repeat steps 2 and 3 until the cluster centers no longer change or change very little.

In this experiment, the chemical components are used as the input feature, and the wine class is the output target. A K-means clustering model is built and trained on the training data, and its prediction performance is validated on the test set.

1.4 Experiment Environment

- Python 3.10.16
- numpy 1.26.4
- pandas 2.2.3
- matplotlib 3.10.0
- scikit-learn 1.6.1

1.5 Experiment Steps

- (1) Load data
 - (2) Separate feature variables (X) and target variable (y) – features are Column 2 to Column 14, target is Column 1 (`Wine`)
 - (3) Split dataset: training set (**70%**) and test set (**30%**)
 - (4) Data preprocessing – feature standardization (**StandardScaler**), dimension reduction (**PCA**)
 - (5) Build and train k-means model (**set n_clusters**)
 - (6) Test and evaluate the machine learning model (e.g., **silhouette score**, **adjusted rand index**)
 - (7) Visualize experimental results
-

2. Experiment Requirements

2.1 Data and Result Visualization

- Draw a scatter plot of the training set after dimensionality reduction (2D).
- Draw the clustering result plot for the training set (showing centroids).
- Draw the WCSS elbow curve and the silhouette coefficient curve, showing the variation of both metrics for the number of clusters from 2 to 10, and indicate the optimal number of clusters.

```
In [14]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (10, 6)

data_candidates = [
    Path("wine.csv"),
    Path("PJJ/Lab08/wine.csv"),
    Path.cwd() / "wine.csv",
    Path.cwd() / "PJJ/Lab08/wine.csv",
]
for candidate in data_candidates :
    if candidate.exists():
        csv_path = candidate
        break
else:
    raise FileNotFoundError("Cannot find wine.csv in the notebook directory or w

df = pd.read_csv(csv_path)
print(f"Loaded dataset from: {csv_path}")
print(df.shape)
df.head()
```

```
Loaded dataset from: wine.csv
(178, 14)
```

Out[14]:

	Wine	Alcohol	Malic.acid	Ash	Acid	Mg	Phenols	Flavanoids	Nonflavanoid.p
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	



```
In [15]: X = df.iloc[:, 1:]
y = df.iloc[:, 0].astype(int)

feature_names = X.columns.tolist()
class_names = sorted(y.unique().tolist())

print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")
print(f"Classes: {class_names}")
```

Feature matrix shape: (178, 13)
Target vector shape: (178,)
Classes: [1, 2, 3]

```
In [16]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y,
)

print(f"Training set: {X_train.shape}, {y_train.shape}")
print(f"Test set: {X_test.shape}, {y_test.shape}")
```

Training set: (124, 13), (124,)
Test set: (54, 13), (54,)

```
In [17]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Standardization complete.")
```

Standardization complete.

```
In [18]: pca = PCA(n_components=2, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

train_pca_df = pd.DataFrame(X_train_pca, columns=["PC1", "PC2"])
train_pca_df["Wine"] = y_train.to_numpy()

print(f"Explained variance ratio: {pca.explained_variance_ratio_.sum():.4f}")

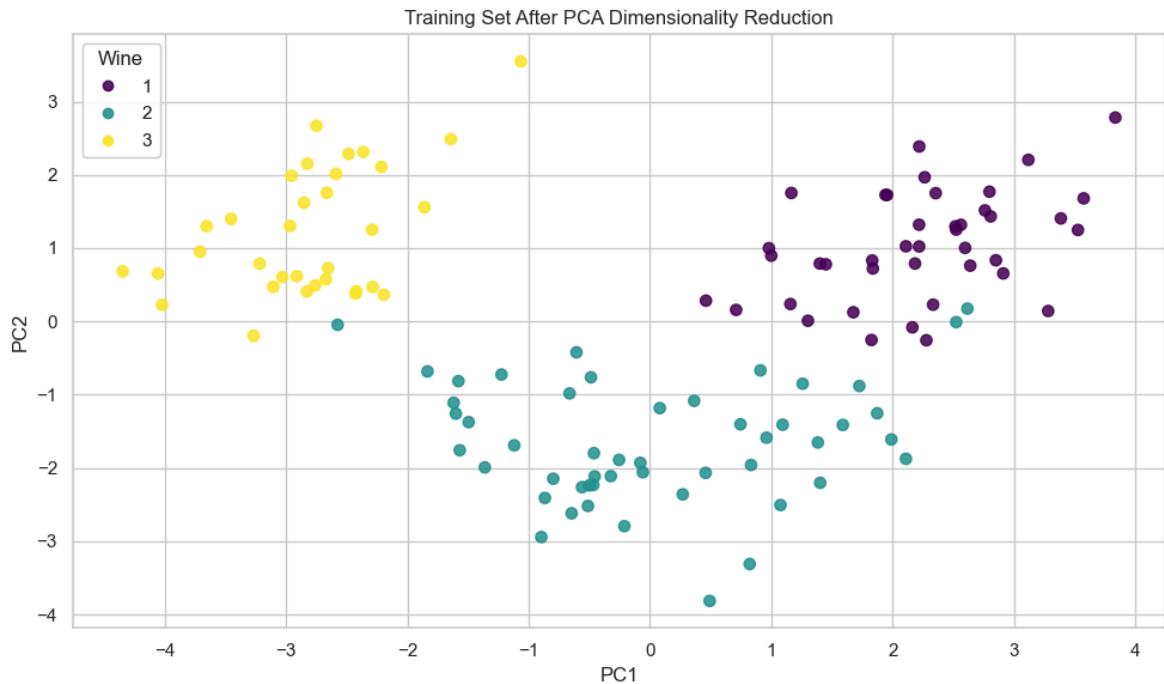
plt.figure(figsize=(10, 6))
scatter = plt.scatter(
    train_pca_df["PC1"],
```

```

train_pca_df["PC2"],
c=train_pca_df["Wine"],
cmap="viridis",
s=45,
alpha=0.85,
)
plt.title("Training Set After PCA Dimensionality Reduction")
plt.xlabel("PC1")
plt.ylabel("PC2")
legend = plt.legend(*scatter.legend_elements(), title="Wine", loc="best")
plt.gca().add_artist(legend)
plt.tight_layout()
plt.show()

```

Explained variance ratio: 0.5494



```

In [19]: n_clusters = 3
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=20)
kmeans.fit(X_train_scaled)

print(f"KMeans model trained with n_clusters={n_clusters}")

```

KMeans model trained with n_clusters=3

```

In [20]: train_cluster_labels = kmeans.labels_

print("Training cluster label counts:")
for cluster_id in range(n_clusters):
    count = int(np.sum(train_cluster_labels == cluster_id))
    print(f"Cluster {cluster_id}: {count}")

```

Training cluster label counts:
Cluster 0: 35
Cluster 1: 43
Cluster 2: 46

```

In [21]: train_silhouette = silhouette_score(X_train_scaled, train_cluster_labels)
train_ari = adjusted_rand_score(y_train, train_cluster_labels)

```

```
print(f"Training Silhouette Score: {train_silhouette:.4f}")
print(f"Training Adjusted Rand Index: {train_ari:.4f}")
```

Training Silhouette Score: 0.2892

Training Adjusted Rand Index: 0.9011

```
In [22]: test_cluster_labels = kmeans.predict(X_test_scaled)
test_silhouette = silhouette_score(X_test_scaled, test_cluster_labels)
test_ari = adjusted_rand_score(y_test, test_cluster_labels)

print(f"Test Silhouette Score: {test_silhouette:.4f}")
print(f"Test Adjusted Rand Index: {test_ari:.4f}")
```

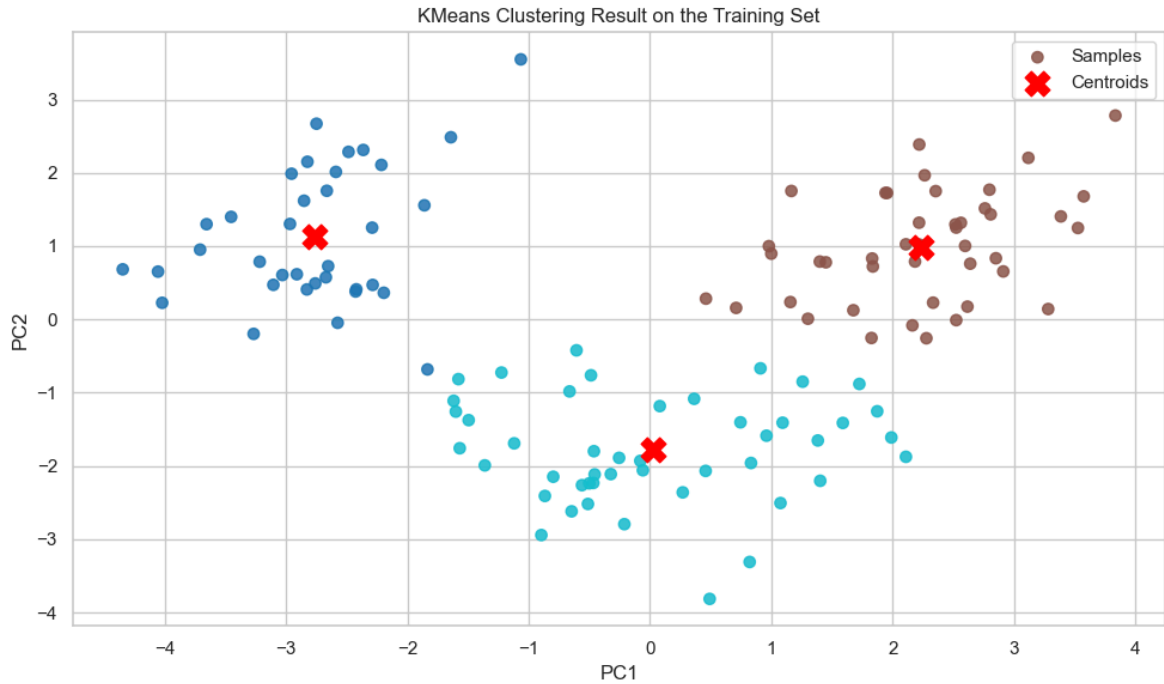
Test Silhouette Score: 0.2658

Test Adjusted Rand Index: 0.8342

```
In [23]: def plot_cluster_result(model, title):
cluster_labels = model.labels_
centers_pca = pca.transform(model.cluster_centers_)

plt.figure(figsize=(10, 6))
plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=cluster_labels,
    cmap="tab10",
    s=45,
    alpha=0.85,
    label="Samples",
)
plt.scatter(
    centers_pca[:, 0],
    centers_pca[:, 1],
    c="red",
    marker="X",
    s=220,
    label="Centroids",
)
plt.title(title)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.tight_layout()
plt.show()

plot_cluster_result(kmeans, "KMeans Clustering Result on the Training Set")
```



```
In [24]: cluster_range = range(2, 11)
inertias = []
silhouette_values = []

for k in cluster_range:
    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_train_scaled)
    inertias.append(model.inertia_)
    silhouette_values.append(silhouette_score(X_train_scaled, labels))

optimal_k = list(cluster_range)[int(np.argmax(silhouette_values))]
print(f"Optimal number of clusters based on silhouette score: {optimal_k}")
```

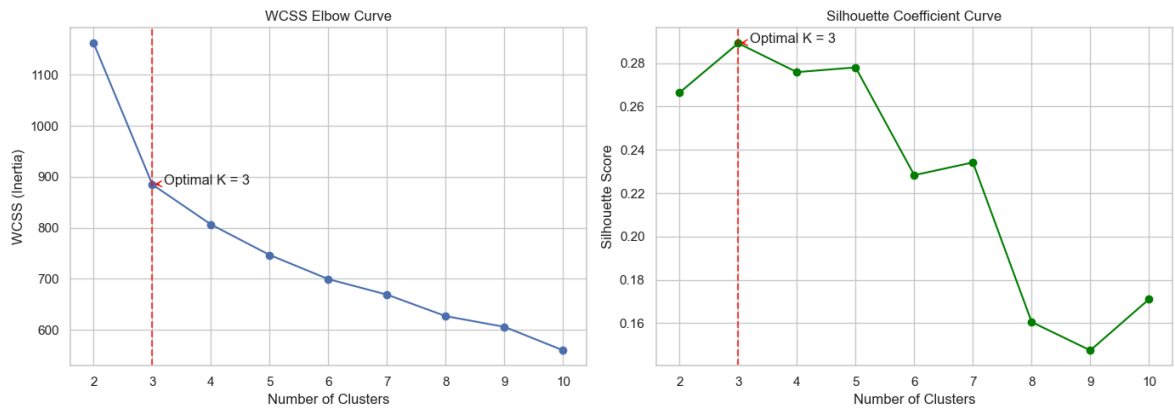
Optimal number of clusters based on silhouette score: 3

```
In [25]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(list(cluster_range), inertias, marker="o")
axes[0].set_title("WCSS Elbow Curve")
axes[0].set_xlabel("Number of Clusters")
axes[0].set_ylabel("WCSS (Inertia)")
axes[0].axvline(optimal_k, color="red", linestyle="--", alpha=0.7)
axes[0].annotate(f"Optimal K = {optimal_k}", xy=(optimal_k, inertias[optimal_k] -

axes[1].plot(list(cluster_range), silhouette_values, marker="o", color="green")
axes[1].set_title("Silhouette Coefficient Curve")
axes[1].set_xlabel("Number of Clusters")
axes[1].set_ylabel("Silhouette Score")
axes[1].axvline(optimal_k, color="red", linestyle="--", alpha=0.7)
axes[1].annotate(f"Optimal K = {optimal_k}", xy=(optimal_k, silhouette_values[optimal_k]

plt.tight_layout()
plt.show()
```



```
In [26]: if optimal_k != n_clusters:
    final_kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=20)
    final_kmeans.fit(X_train_scaled)
    final_train_labels = final_kmeans.labels_
    final_test_labels = final_kmeans.predict(X_test_scaled)

    final_train_silhouette = silhouette_score(X_train_scaled, final_train_labels)
    final_train_ari = adjusted_rand_score(y_train, final_train_labels)
    final_test_silhouette = silhouette_score(X_test_scaled, final_test_labels)
    final_test_ari = adjusted_rand_score(y_test, final_test_labels)

    print("The current model is not optimal, so the model is retrained with the")
    print(f"Final Training Silhouette Score: {final_train_silhouette:.4f}")
    print(f"Final Training Adjusted Rand Index: {final_train_ari:.4f}")
    print(f"Final Test Silhouette Score: {final_test_silhouette:.4f}")
    print(f"Final Test Adjusted Rand Index: {final_test_ari:.4f}")
    plot_cluster_result(final_kmeans, f"Optimal KMeans Clustering Result (K={optimal_k})")
else:
    print("The current model already uses the optimal number of clusters.")

print(f"Conclusion: the silhouette-based optimal cluster count is {optimal_k}, and the baseline model uses {n_clusters}.
```

The current model already uses the optimal number of clusters.

Conclusion: the silhouette-based optimal cluster count is 3, and the baseline model uses 3.